

Developer Series 3

Agentic Engineering Workflow & Automated SDLC — Draft 2 / Final Workflow Specification

Purpose. Convert the Draft 1 agent architecture into an executable operating model across ChatGPT, Cursor, VS Code, GitHub, CI/CD and deployment. This draft defines triggers, permissions, handoffs, retries, branch/worktree rules, automated commits/PRs, verification gates and escalation boundaries.

Design objective. Routine implementation and ordinary engineering failures should progress through automated build, review, repair and verification loops without requiring the operator to manually diagnose each error. Human authority remains required for risky, ambiguous or privileged decisions.

Architect → Atlas/Cursor → Guardian + Medic → GitHub PR → Sentinel/VS Code → Watchtower + Resolver → Harbor → CI/CD → deployment verification

1. Control plane and authority

Layer	Controller	Authority
Architecture	Architect (ChatGPT)	Feature Contracts, architecture decisions, cross-system requirements; cannot silently override repository truth
Build	Atlas (Cursor)	Decompose/assign/integrate implementation; cannot merge own PR or waive gates
Build reliability	Guardian + Medic	Workflow completeness + runtime/debug recovery; cannot redefine product architecture
Verification	Sentinel (VS Code)	Dispatch independent review and decide verification state; cannot waive mandatory security/data gates
Verification reliability	Watchtower + Resolver	Audit gate execution + root-cause/recovery; cannot falsify/ignore failed evidence
Release	Harbor + GitHub Actions	Merge/release/deploy only after required green checks
Durable state	GitHub + .vossie/	Issues, PRs, ADRs, contracts, status, evidence and handoffs

Human-only boundaries

Human approval is required for destructive/high-risk production migrations, new secrets/credentials, privileged third-party access, security exceptions, architectural decisions that conflict with locked ADRs, ambiguous product decisions, irreversible data actions, production payment changes, or a repair loop that exhausts its retry budget.

2. Trigger and feature-intake workflow

Trigger A — new feature: approved product requirement, screen/flow from the Figma/177-screen map, bug/issue, or explicit architecture task creates/updates a GitHub issue and a .vossie Feature Contract.

Trigger B — defect: CI, runtime, monitoring, QA or human report creates a defect work item with reproduction evidence and links to the affected feature/PR.

Trigger C — design change: accepted Figma/design-system change updates the design reference/version and opens implementation work only for affected components/screens.

Required Feature Contract

Field	Required content
feature_id	Stable identifier used in branch, issue, status and handoffs
goal / non-goals	What must change and what must not
affected surfaces	Buyer/Seller/Courier/Operations + screen/component IDs
acceptance criteria	Observable functional and UX outcomes
contracts	API/data/events/authZ/integration expectations
design refs	Figma frame/component/version references where applicable
risk class	low / normal / high; determines required gates
tests required	unit/integration/E2E/security/accessibility/data checks
dependencies	features, migrations, providers, assets, blocked decisions

Architect marks the contract READY only when enough information exists for implementation. Atlas must not invent unresolved product/architecture decisions to keep the pipeline moving.

3. Cursor execution — Atlas and sub-agents

Atlas reads the Feature Contract, relevant ADRs, .vossie agent rules, Figma context and current repository state. It decomposes work into scoped tasks and assigns only the required specialists.

Agent	Allowed scope	Required output
Pixel	React/UI, adaptive layouts, design-system use	UI code + accessibility + relevant tests
Forge	Server/API/domain services	Typed server code + validation + tests
Schema	DB schema/migrations/RLS	Migration + rollback notes + RLS/data tests
Market	Marketplace domain	Listings/Requests/order logic + tests
Relay	Courier/hub/transfer/maps/custody	State logic + integration tests
Bridge	Provider adapters/auth/payments/notifications	Adapter implementation + sandbox/error tests
Kinetic	Motion.dev/microinteractions	Motion code + reduced-motion/performance checks
Probe	Implementation tests	Missing unit/integration coverage + reproducible results
Scribe	Technical state/docs	Contract/ADR/status/docs updates

Concurrency

Independent tasks may run in parallel using isolated branches/worktrees. Tasks touching the same migration, core contract or high-conflict files are serialized. Atlas owns integration order. Specialists may not merge each other's work or silently edit outside assigned scope.

Local completion gate

Before Atlas opens a PR: formatting, lint, typecheck, affected unit/integration tests and production build must pass. Guardian then checks workflow completeness; Medic verifies execution and enters repair mode for failures.

4. Reliability loop — Guardian and Medic

Guardian audits orchestration: required tasks assigned, dependency order valid, no skipped deliverables, tests/evidence present, contract/status updated, and Atlas has not crossed a permission boundary.

Medic owns executable health: dependency/install failures, imports, TypeScript/compiler errors, runtime exceptions, build failures, broken API wiring and failing implementation tests.

Automatic recovery algorithm

Capture → classify → reproduce → isolate → identify owner → repair → rerun targeted check → rerun regression subset → record evidence → Guardian re-audit.

Retry policy

Use a bounded retry budget. Default: up to 3 materially different repair attempts for an ordinary deterministic code failure. Repeating the same unsuccessful edit does not count as progress. After exhaustion, Medic produces a structured failure packet and escalates through Atlas/Architect or to the human boundary when required.

Failure packet

Feature/commit; failing command/test; exact error summary; reproduction steps; suspected root cause; files/components involved; attempted fixes and outcomes; remaining hypotheses; risk of further automated changes; recommended next action.

5. Git, branch, commit and PR automation

Object	Convention / automation
Branch	feature/-, fix/-, chore/-; never autonomous direct work on main
Worktree	One isolated worktree per parallel agent task when file overlap is low
Commit	Atomic conventional commits: feat:, fix:, test:, refactor:, docs:, chore:, perf:, ci:
Push	Atlas automation pushes only its feature/fix branch after local gate
PR	Auto-created with Feature Contract link, summary, affected screens/services, tests, migrations, risk and evidence
Main protection	Required checks/reviews; no force push; no self-merge by Atlas
Merge	Harbor permits merge only after current required checks are green; squash/rebase policy fixed repo-wide
Release record	Merged feature/issue, evidence, deployment and smoke-test result written back to GitHub/.vossie

Automation must preserve traceability. Agents may create branches, commits, pushes and PRs automatically, but must never hide failed checks, fabricate evidence, rewrite protected history or use an automated commit to bypass a review gate.

6. VS Code independent verification

Sentinel receives the PR only after Cursor's local gate is green. It dispatches verification based on the Feature Contract and risk class.

Agent	Gate	Pass evidence
Lens	Code/architecture	Diff reviewed against contracts/ADRs; maintainability issues resolved
Aegis	Security	AuthZ, secrets, dependencies, input boundaries and relevant threat checks pass
Vector	QA/E2E	Required Playwright/scenario/regression tests pass
Pulse	Performance/a11y	Responsive, reduced-motion/a11y and applicable performance checks pass
Vault	Data	Migration, RLS, integrity, transaction/rollback expectations pass
Watchtower	Workflow audit	All required gates ran on current commit; evidence is current and complete
Resolver	Recovery	Any verification failure has root-cause packet and validated remediation loop

Verification failures route back to Atlas/Medic when implementation changes are required. Sentinel/Resolver may repair verification infrastructure within their allowed scope, but must not rewrite product implementation merely to manufacture a passing result.

7. CI/CD and GitHub Actions gates

Stage	Automated gates
PR fast gate	Install/cache integrity; formatting; lint; typecheck; unit tests; affected integration tests; build
Security gate	Secret scan; dependency/vulnerability scan; static/security checks appropriate to stack
Data gate	Migration validation; schema/RLS checks where changed; test database apply/rollback where feasible
E2E gate	Playwright critical flows on preview/staging; role/authZ and affected user journey
UX gate	Accessibility checks; reduced-motion path; responsive smoke checks; visual regression where configured
Merge gate	All required current checks + approvals + Watchtower completeness state
Deploy gate	Vercel/Supabase staging or configured target; migrations in controlled order
Post-deploy	Health endpoint/app boot; critical smoke flow; error/log check; deployment status recorded

A failed post-deployment smoke test blocks promotion and starts Resolver/Medic recovery. Where deployment mechanics support safe automated rollback, Harbor may roll back to the last verified release. Database rollback is never assumed safe; destructive/data-loss risk escalates.

8. Machine-readable handoff contract

Every agent handoff should be structured rather than conversational-only. Minimum schema:

task_id • feature_id • from_agent • to_agent • repo_commit • scope • files_changed • contracts_touched • tests_run • test_results • known_risks • open_failures • evidence_refs • next_action • status.

Recommended .vossie workspace

architecture/ ADRs and locked technical rules; **agents/** agent contracts and permissions; **features/** Feature Contracts; **contracts/** API/data/event schemas; **screens/** screen registry; **design/** Figma/design references; **security/** security requirements; **handoffs/** structured agent handoffs; **status/** current feature/build/release state.

Status model

PLANNED → READY → BUILDING → CURSOR_VERIFYING → PR_OPEN → VS_VERIFYING → REPAIRING (when needed) → MERGE_READY → MERGED → DEPLOYING → DEPLOYED → VERIFIED. BLOCKED and HUMAN_REQUIRED are explicit terminal/holding states until the blocker is resolved.

9. Final end-to-end operating workflow

- 01 Intake.** Requirement/design/bug enters GitHub.
- 02 Contract.** Architect produces or validates Feature Contract and risk class.
- 03 Plan.** Atlas reads repo state and decomposes work.
- 04 Isolate.** Branch/worktree(s) created automatically.
- 05 Build.** Required Cursor specialists implement in scoped parallel/serial tasks.
- 06 Implement-test.** Probe and specialist tests execute continuously.
- 07 Workflow audit.** Guardian checks delegation, dependencies, scope and completeness.
- 08 Runtime verification.** Medic builds/runs and repairs ordinary failures within retry budget.
- 09 Integrate.** Atlas integrates green tasks and reruns local completion gate.
- 10 Commit/push.** Atomic conventional commits pushed automatically.
- 11 PR.** PR generated with contract, risk, migration and evidence metadata.
- 12 Independent review.** Sentinel dispatches Lens/Aegis/Vector/Pulse/Vault as required.
- 13 Verification audit.** Watchtower confirms all current required gates executed.
- 14 Recovery.** Resolver diagnoses failures; implementation defects route to Atlas/Medic; loop repeats.
- 15 Merge gate.** Harbor confirms approvals and green CI; only then merges.
- 16 Deploy.** CI/CD deploys controlled preview/staging/target release.
- 17 Post-deploy verification.** Vector/health checks execute critical smoke paths.
- 18 Close.** .vossie status, issue, PR, docs and release evidence update automatically.

Operating outcome

The operator should normally interact at the requirement/decision level, not as the first-line debugger. Ordinary code errors are expected to be caught, diagnosed and retried inside the system. The workflow escalates when human authority or judgment is genuinely required rather than pretending every failure can safely be automated away.

10. Draft 2 lock summary

Locked: Atlas and Sentinel remain separate build/control leads; Guardian + Medic and Watchtower + Resolver provide two-layer reliability; GitHub/.vossie is shared durable state; work is branch/worktree isolated; conventional atomic commits and PRs are automated; no autonomous direct-to-main development; mandatory CI/security/data/QA gates cannot be bypassed; ordinary failures enter bounded repair loops; risky/privileged/ambiguous decisions escalate; deployment requires post-deploy verification.

Implementation artifacts to create when the repository is connected

Agent instruction files/prompts • .vossie directory/schema • GitHub issue and PR templates • CODEOWNERS/protection rules where appropriate • GitHub Actions workflows • branch/worktree scripts • commit/PR automation • test commands • Playwright projects • security/secret scanning configuration • migration verification scripts • deployment/rollback scripts • structured handoff/status schemas.

Relationship to previous series

Developer Series 1 defines the application architecture, stack and multi-agent principle. Developer Series 2 defines the Figma/UI/UX/motion production pipeline. Developer Series 3 defines how the engineering agents actually execute, verify, repair, commit, merge and release that work.